Minesweeper Maintenance

EECS 581, Group #7
Team Members: Sean Crosby, Thad Flournoy, Eyassu Mongalo, Lillian Brooks-Kanost, Caleb Neil, Nick Paslay

1. Introduction

1.1. Overview

The goal for this project was to enhance a version of minesweeper created by another team. The main goals of this enhancement were to add a user interface to improve the experience of playing the game, to add a timer element and a leaderboard to give players a higher-level goal, and to add an AI solver algorithm. The version of Minesweeper being updated was built entirely in Python with no graphics package, so the game state and user input was handled by the command line. The maintenance aims to port the functionality of that command-line based game to a GUI and add the features with as few unnecessary changes to the original logic as possible.

1.2. Team Organization

- UI / Graphics, Scrum Master - Nick Paslay
- AI Solver - Lillian Brooks-Kanost & Eyassu Mongalo
- Timer Addition - Caleb Neil
- Cell Chording, Documentation (UML Diagram) - Thad Flournoy
- UI / Graphics, Documentation (Design Doc) - Sean Crosby

1.3. Hours Invested

Hours per task are self-reported by team members. Each team member contributed an additional 1-2 hours of code review. Estimations were decided by our scrum master.

Timeline:

September 27th: Sean Crosby (2 hours)

September 28th: Sean Crosby (2 hours), Lillian Brooks-Kanost (1.5 hours), Caleb Neil (0.5 hours)

September 29th: Thad Flournoy (2 hours), Lillian Brooks-Kanost (0.5 hours), Eyassu Mongalo (3 hours), Caleb Neil (0.5 hours)

September 30th: Caleb Neil (2 hour), Lillian Brooks-Kanost (0.5 hours), Nick Paslay (2 hours)
October 1st: Thad Flournoy (2 hours), Lillian Brooks-Kanost(0.5 hours), Nick Paslay (3 hour)
October 5th: Sean Crosby (2 hour), Caleb Neiil (1.5 hours), Lillian Brooks-Kanost (0.5 hours)
October 6th: Lillian Brooks-Kanost (0.5 hours), Eyassu Mongalo (2 hours)
October 7th: Eyassu Mongalo (1 hour), Lillian Brooks-Kanost (0.5 hours)
October 9th: Lillian Brooks-Kanost (1.5 hours)
October 10th: Sean Crosby (0.5 hours), Lillian Brooks-Kanost (2.5 hours)
October 11th: Eyassu Mongalo (1 hours), Thad Flournoy (1 hour)

- Scrum Master / Project Organization - Nick Paslay
    - Expected: 2 hours, Actual: 2 Hours
- UI / Graphics Setup - Sean Crosby
    - Expected: 3 hours, Actual: 2.5 hours
- AI Solver (Easy) - Lillian Brooks-Kanost and Eyassu Mongalo
    - Lillian: Expected: 1 hour, Actual: 1 hour
    - Eyassu: Expected: 1 hour, Actual: 1 hour
- AI Solver (Medium) - Lillian Brooks-Kanost and Eyassu Mongalo
    - Lillian: Expected: 1 hour, Actual: 1 hour
    - Eyassu: Expected: 1 hour, Actual: 2 hours
- AI Solver (Hard) - Lillian Brooks-Kanost and Eyassu Mongalo (2 hrs)
    - Lillian: Expected: 1 hour, Actual: 2 hours
    - Eyassu: Expected: 1 hour, Actual: 2 hours
- AI Solver UI - Eyassu Mongalo (3 hour)
    - Expected: 3 hours, Actual: 3 hours
- Timer Addition - Caleb Neil (2 hour)
    - Expected: 2 hours, Actual: 2 hours
- Timer UML Diagram - Thad Flournoy (3 hours)
    - Expected: 3 hours, Actual: 1 hours
- Cell Chording - Thad Flournoy (3 hours)
    - Expected: 3 hours, Actual: 3 hours
- UI / Graphics Win/Loss Conditions - Nick Paslay
    - Expected: 2 hours, Actual: 3 hours
- UI Input Handing - Sean Crosby
    - Expected: 1 hours, Actual: 1 hours
- Design Doc - Sean Crosby
    - Expected: 2 hours, Actual: 2 hours

2. Addition Overview

2.1. UI / Graphics

As the original version of this project used the command line for all input and display, a key point of maintenance was adding a GUI to make the game easier to play and test. The PyQt5 package was chosen for its compatibility with the KU Cycle Servers. Much of the original code from the main function was split between two window classes, which the main loop now initializes. First, a StartGameDialogue class is used to take input from the user on the amount of mines they would like to play with and the AI solver mode. After that window is submitted, a GameWindow class is used to encapsulate all of the display updating and input handling for the main game.

2.2. AI Solver

For the updated version, an AI solver was implemented with three different intelligence levels and two solving modes. The “Interactive” mode will alternate turns between the user and the AI, while the “Automatic” mode lets the AI take turns until it completes the board. In the “Easy” setting, cells are uncovered randomly. On “Medium”, the AI will flag tiles if they are the only possible bomb spots of an uncovered tile (e.g. a blank tile showing 1 with 1 hidden tile in its proximity will have that hidden tile flagged), and tiles will be uncovered if the uncovered tile indicates that all surrounding tiles have already been flagged. On “Hard”, an additional 1-2-1 pattern rule is applied where if uncovered tiles containing 1, 2, and 1 appear next to each other, it can be assumed that the outer neighbors are mines and the inner neighbor is safe. On medium and hard, a random click is used when no rule can be applied.

2.3. Timer

A timer was added to track the time it takes for a player to finish the puzzle on various numbers of starting bombs. For each possible starting bomb amount ([10,20]), a “Best Time” value will be stored in a leaderboard.json file so that the player can try again to beat their time.

3. Detailed Design

ai_solver

AiSolver.__init__

- Responsibilities: initialize solver global variables
- Interface:
    - Inputs: difficulty
    - Outputs: N/A

    - Error Handling: Any difficulty passed in must exist in the Difficulty enum, so improper input would throw a type error

AiSolver.generate_changes

- Responsibilities: Individual step in the AI solving process, used for all difficulty levels
- Interface:
    - Inputs: board
    - Outputs: Sets of cells to uncover and flag
    - Error Handling:
- Description: For every cell on the board, check the state (covered / uncovered / flagged). If a cell is covered, it should be added to the fallback set, which a fallback cell will be selected from and uncovered if no other rules are applied (or if the mode is "Easy", where no rules exist). If the user has the AI set to "Medium" or "Hard", the solver should add cells to "uncover" and "flag" arrays based on their rules. If any cells exist in either of these sets, then no fallback cell is chosen and the uncover/flag actions are performed to relevant tiles by the game logic function.

board.py

Board.get_potential_1_2_1

- Responsibilities: return 3 lists of coordinates for unflagging, uncovering, & flagging if a given uncovered cell has two adjacent uncovered "1" cells
- Interface:
    - Inputs: row, col
    - Outputs: ([(int, int)], [(int, int)], [(int, int)])
    - Error Handling: If the row/col are outside of the board dimensions, the function will always return empty lists
- Description: For both vertical and horizontal directions, check whether the adjacent cells contain 1 adjacent mine. If they don't, set a variable to false. If the variable is true for either direction, then the "2" tile has adjacent "1"s and therefore applies to the 1-2-1 rule so it sets the inner spots to be unflagged if they are as well as uncovered if they're not already uncovered. The corners are checked and if one corner is uncovered that guarantees that the reflection along the vertical horizontal 1-2-1 will exist & be a mine so they get flagged.

display.py

CellButton.mousePressEvent

- Responsibilities: Decide whether a tile's input event was a left click (reveal) or right click (flag)

- Interface:
    - Inputs: event
    - Outputs: Emit Signal

StartGameDialogue._initialize_ai_prompt

- Responsibilities: Create prompt for AI mode and difficulty level for the init function of StartGameDialogue.
- Interface:
    - Inputs: self
    - Outputs: none
    - Error Handling: no user input to this function is possible, so errors are unlikely
- Description: Uses for loops instead of hardcoded options for more readable code. Adds elements directly to the display window rather than returning them to the init function. Difficulty buttons are initialized with setVisible(False) and only displayed after an AI mode has been chosen.

StartGameDialogue.check_bomb_input

- Responsibilities: Error check for a valid bomb input
- Interface:
    - Inputs: input_text
    - Outputs: num_mines
- Description: If the input_text is a number between 10 and 20, then that number is returned as an integer. Any other input will result in an error message and “None” being returned instead

StartGameDialogue._set_visibility

- Responsibilities: Hide/show the AI Difficulty buttons
- Interface:
    - Inputs: value
    - Outputs: N/A
- Description: If the Mode value is set to NONE, the AI buttons are set to be invisible. Any other Mode value will result in the buttons being displayed.

GameWindow.__init__

- Responsibilities: Initialize the game environment and display all game elements
- Interface:
    - Inputs: num_mines, ai_mode, ai_difficulty
    - Outputs: N/A

    - Error Handling: All inputs are validated prior to entering this function
- Description: Creates all elements of the game window including the flag count, the game state readout, the timer, the board, and the grid labels. Initializes an instance of the Game class to handle all game logic.

GameWindow.update_ui

- Responsibilities: Change the GUI to represent the state of the board after a change has been made
- Interface:
    - Inputs: self.board
    - Outputs: N/A
    - Error Handling: If a cell reaches an invalid state, a ValueError will be thrown and the window will close
- Description: Goes through each board cell and reads its state. Pulls in style variables from the styles.py file and applies them based on the state of the cell. For numbered cells, an additional color attribute is added to the style based on the number to give a visual indication of how many bombs are nearby.

game_logic.py

Game.chord_cell

- Responsibilities: Uncover cells around an uncovered cell if all bombs have been flagged
- Interface:
    - Inputs: row, col
    - Outputs: N/A
    - Error Handling: It is assumed that for this function to be called, a valid tile must have been clicked
- Description: For a given uncovered cell, check the state of all of its neighbors. If the number of neighbors which are bombs matches the number that are flagged, then all covered neighbors should be uncovered.

Game.start_timer + Game.get_timer

- Responsibilities: Manage the timer element for the game
- Interface:
    - Inputs: N/A
    - Outputs: N/A
    - Error Handling: Return “00:00:00” if an invalid elapsed time value exists
- Description: get_timer returns the amount of time that has elapsed since start_timer was called.

leaderboard.py

update_leaderboard

- Responsibilities: Update the leaderboard after a victory
- Interface:
    - Inputs: n_bombs, time, mode
    - Outputs: leaderboard.json
- Description: Updates the leaderboard.json file with an entry based on the number of bombs and whether or not the AI assist was used. For each coupling of bomb counts and AI assist values, a different best time is recorded. If this time is beaten by the current time, that value is replaced in the json

5. Performance and Scalability

- Expected Load: The maximum load for this program is one user playing with the interactive solver on a 10x10 board. Under this load, the program loads and responds to input in an insignificant amount of time
- Limitations: This program is only meant for a restricted 10x10 grid size, with no scalability for choosing a size of board, and meant for 10-20 bombs on the board.

6. Program Environment

- Environments: Windows, Linux, MacOS
- Language: Python
- Required Packages: PyQt5

7. Testing Strategy

- Unit Testing: We tested the start and end game screen, bombs, flags, tiles with a right click for flag, left click for choosing a tile, all separately as we wrote the code. We tested various AI conditions, including Easy, Medium, and Hard on both the interactive mode and solver mode. Finally, we ensured that best times were accurately stored in the leaderboard.
- Integration Testing: We implemented the graphics and tested to make sure that all old game logic properly functioned in the new environment.

- End-to-End Testing: We tested launching the application, the end game screen, the start screen, changing numbers of bombs, choosing an AI solver mode, starting and ending the timer, and loading the board all separately and then all together.
- Quality Metrics: Another member of the team had to review and merge the branch of a member when coding, and test cases were used to credit error codes and consistency.